CONTENTS

Document Intelligence — Engineering Design Spec

Date: 2026-06-24 · **Status:** Draft for review · **Owner:** Nilendu **Companion docs:** requirements & interpretation log · retrieval API additions

---

1. Summary & goals

A unified document-intelligence platform that turns a bank client's KYC documents (PDF / DOCX / JPEG / PNG) into a **versioned, per-client knowledge tree** and serves it to downstream services via an API for search, single-document Q&A, and structured fact retrieval — with PII-safe processing throughout.

Primary goal: every document a client submits becomes a **knowledge subtree** (the novel data structure, §6) hanging under `client → doc-type → version`, and facts are consolidated into a **client-level merged view** (§7). Downstream consumers query by `client_id` and traverse.

Geography/languages: US, Canada, Mexico · English + Spanish (bilingual docs supported).

2. Non-goals (v1 boundaries)

Entity resolution for the primary key (`client_id` arrives with the doc); risk scoring; PEP / sanctions adjudication; **cross-client** AML link analysis; automated beneficial-owner graph walking for persona-moral files (we capture + link, we do not adjudicate); a graph DB; a human-review UI (we emit `needs_review` flags + a list endpoint); full bitemporal audit; any local Azure DI Docker container. document_intelligence holds **no** Stellar/COIN/VDI credentials.

3. Architecture

Three concerns, one service:

1. **Ingestion pipeline** — OCR → language split → PII-safe classification gate → dual extraction → knowledge-subtree build → accessibility-rep generation → cross-document merge → versioning (§5).
2. **Storage** — one `ltree` forest in Postgres + pgvector, partitioned + RLS-isolated by `client_id` (§4).
3. **Serving API** — per-client tree traversal, hybrid scoped search, provenance, capabilities manifest, answerable-questions, masked projections, version deltas (§8).

External dependencies: Azure AI Vision **Read** OCR (cloud); the **`retrieval`** service as the model gateway (`/api/embed` , `/api/llm/complete` , `/api/rerank` , `/api/models` — see companion doc; URLs + `X-API-KEY` via env); Postgres 16 + pgvector + `ltree` ; S3/MinIO for source files.

(See the three rendered diagrams in the brainstorm: two-layer architecture; the `knode` / `arep` subtree; the PII-gate + dual-extraction flow.)

4. Data model (Postgres 16 + pgvector + ltree)

Conventions mirror `retrieval/backend`: sentinel `vector.` schema rewritten at runtime; idempotent startup migrations (`CREATE ... IF NOT EXISTS` + DO-block guards); `BIGSERIAL` / `uuid` keys; soft delete (`deleted_at`) + partial indexes; JSONB + GIN for flexible payloads; generated `tsvector` + GIN for FTS; **vector columns added at runtime** once dim is discovered (HNSW `vector_cosine_ops`); writes wrapped in best-effort `SET ROLE`. New extension migration adds `CREATE EXTENSION IF NOT EXISTS ltree`.

Multi-tenancy: every table carries `client_id`; tables `PARTITION BY HASH (client_id)` (fixed count, e.g. 64), each partition with its own HNSW index; **RLS `FORCE`d** with `client_id = current_setting('app.current_client_id')`, bound per asyncpg pool-acquire and RESET on release.

TABLES

- **`di_documents`** — one row per source file: `id uuid`, `client_id`, `document_name`, `s3_uri`, `sha256`, `mime`, `doc_type`, `doc_category`, `subject`, `jurisdiction`, `lang_profile jsonb`, `sensitivity_bucket`, `gate_decision`, `confidence`, `ocr_engine`, `page_count`, `ocr_text`, `ocr_lines jsonb` (per-line text+bbox+confidence, for deterministic extraction), timestamps, `deleted_at`.
- **`doc_version`** — `id uuid`, `client_id`, `doc_id`, `version_no`, `content_hash` (dedup), `supersedes`, `is_current` (partial-unique per `(client_id, doc_id) WHERE is_current`), `changed_fields jsonb`, `created_at/by`.
- **`knode`** — the returnable nodes (§6): `id uuid`, `client_id`, `doc_id`, `version_id`, `parent_id`, `path ltree`, `node_type` (`document|section|chunk|table|figure|fact|summary`), `seq`, `depth`, `title`, `content`, `content_tsv` (generated), `context_prefix`, `content_embedding vector(D)` (runtime), `cross_refs uuid[]`, `entity_ids uuid[]`, `attribute_key` (canonical key for `fact` nodes, e.g. `identity.date_of_birth`), `value_text/value_date/value_num`, `verification_status` (`checksum_verified|gov_verified| llm_unverified|unverified`), `confidence`, `sensitivity` (per-node PII level, for masking), `valid_from` / `valid_to` (real-world fact validity, nullable — powers time-travel alongside the version chain), `provenance jsonb` (page/bbox/offsets/model+version), `token_count`, timestamps, `deleted_at`. Indexes: `gist(path)`, `gin(content_tsv)`, `hnsw(content_embedding)`, `(client_id, doc_id, version_id)`, `(client_id, node_type)`, partial `gin(...)` as needed.
- **`arep`** — accessibility representations (searched, map back to a `knode`): `id`, `knode_id`, `client_id`, `doc_id`, `version_id`, `path ltree`, `rep_type` (`hypothetical_q|proposition|summary|alt_phrasing|synonym_expansion|table_desc|figure_desc|keyword_set|translation`), `rep_lang`, `rep_text`, `rep_tsv` (generated), `rep_embedding vector(D)` (runtime), `gen_model`. Indexes: `hnsw(rep_embedding)`, `gin(rep_tsv)`, `gist(path)`.
- **`client_merged_fact`** — the client-level merged view (§7): `id`, `client_id`, `attribute_key`, `resolved_value`, `value_*`, `confidence`, `conflict` bool, `needs_review`, `source_fact_ids uuid[]` (provenance fan-out to `knode` facts), `updated_at`.
- **`di_entity`** — entities referenced by `knode.entity_ids` (people/orgs/addresses within a client): `id`, `client_id`, `entity_type`, `normalized_name`, `attributes jsonb`.

- `di_decision_trace` — per-document gate audit: classifier output + confidence, PII entities + scores, gate decision, language profile. (Compliance audit.)

5. Ingestion pipeline

Driver `ingest_document(client_id, file)` emits SSE stage events (`upload→ocr→lang→classify→pii→gate→extract→subtree→arep→merge→version`).

1. **Upload** — store source to S3; `sha256`; dedup vs `doc_version.content_hash` (identical re-upload ⇒ no-op).
2. **OCR (Azure Vision Read)** — `ocr_vision.py`: text + per-line bbox + confidence; image-first (rasterize multipage PDFs or use the async Read operation). Never raises (degrades to text layer). Persist `ocr_text` + `ocr_lines`.
3. **Language split** — `lingua-py` (EN/ES): dominant language + per-span languages → `lang_profile`; tiny/low-conf/out-of-scope spans fail safe.
4. **Stage 0 — cheap gate** — compiled regex + anchor gazetteers (EN + ES) + ID-checksum sweep; high-specificity hit short-circuits to a doc_type.
5. **Stage 1 — local classifier** — TF-IDF (char_wb 3–5 + word 1–2) + `LinearSVC` (`CalibratedClassifierCV`); SetFit (`paraphrase-multilingual-MiniLM-L12-v2`) upgrade for hard classes. **No training to launch** (rules + weak supervision; §3.11 of report). UNKNOWN + non-LOW sensitivity ⇒ fail safe.
6. **Stage 2 — PII/sensitivity** — one multilingual Presidio `AnalyzerEngine` (`en_core_web_lg` + `es_core_news_lg`; iterate language spans) + custom MX recognizers (CURP/RFC via `stdnum`, INE Clave de Elector). Sensitivity bucket LOW…CRITICAL; persist to decision trace.
7. **Stage 3 — routing gate** — config-driven pure fn `(doc_type, sensitivity, confidence)` ⇒ `SEND_TO_LLM | REDACT_THEN_SEND | DETERMINISTIC_ONLY`. **v1: gate open** (hooks built); `REDACT_THEN_SEND` implemented but inactive; fail-safe on UNKNOWN.
8. **Extraction (dual):**
   - **LLM path** (`SEND_TO_LLM`) — via retrieval `/api/llm/complete`: base classification + LLM- chosen **attribute KV** extraction → `fact` knodes (`llm_unverified`), structure reconstruction (sections), and the full `arep` aid generation (§6).
   - **Deterministic path** (`DETERMINISTIC_ONLY`) — strict per-jurisdiction pydantic schemas from the OCR dump (§5.1) → `fact` knodes (`checksum_verified`/`gov_verified` where applicable); leaner arep (field + content embeddings only).
9. **Subtree build** — assemble `knode` tree (document→section→chunk/table/figure→fact + synthetic `summary`); structure-aware chunking (semantic-breakpoint split only for over-long blocks); `context_prefix` per node; embeddings via `/api/embed`; reading-order + page provenance.
10. **arep generation** — per node: hypothetical_q, proposition, summary, alt_phrasing/synonyms, table/figure desc, and **EN↔ES translation reps** (cross-lingual); each embedded + FTS-indexed. Async backfill after the synchronous core lands.
11. **Merge** — consolidate `fact` knodes into `client_merged_fact` (§7).

12. **Version** — new `doc_version`; diff vs current; reuse unchanged nodes' embeddings/aids; flip `is_current`.

5.1 DETERMINISTIC EXTRACTION (NO-LLM PATH)

Per-jurisdiction strict schemas; every field emitted as `{value, raw_ocr, source, checksum_ok, confidence, bbox}`; self-validating IDs captured by global regex sweep + checksum.

- **Universal:** Passport ICAO 9303 MRZ (`PassportEye` / `mrz`).
- **US:** SSN/EIN/ITIN (`stdnum.us.*`), state DLs, W-2/1099 anchors.
- **Canada:** SIN/BN (Luhn, `stdnum.ca.*`), province DLs, T4/NOA anchors.
- **Mexico:** CURP (18-char, **hard** check via `stdnum.mx.curp` + state catalog + DOB/sex cross-checks; accept sex `X`), RFC (12/13-char, structure strict, mod-11 **soft**), INE/IFE (Clave de Elector + reverse TD1 MRZ; branch on model D+; no checksum → cross-field reconciliation), SAT CSF (idCIF + QR; only doc with a free gov verify endpoint), comprobante de domicilio (≤3-month recency). Required-doc lists are **config-driven** (regulatory drift).
- Non-checksum fields → bbox label-anchored KV (`rapidfuzz` + nearest right/below + `dateparser` / `usaddress`/libpostal). Lib set: `python-stdnum`, `PassportEye`, `dateparser`, `rapidfuzz`, `pydantic`, `pycountry`.

6. The knowledge subtree (the star)

Two tables — **`knode`** (canonical, returned) + **`arep`** (representations, searched; "index-many / return-parent"). The four required properties: **semantic** (`content_embedding` + every `arep.rep_embedding`), **logical** (`parent_id` + `path` tree, `cross_refs[]` DAG, `entity_ids[]`), **contextual** (`context_prefix` per node), **accessibility** (the open `arep` table). `fact` nodes are first-class.

HEADLINE CAPABILITIES (ALL V1 UNLESS NOTED)

1. **Answer at any altitude** — collapsed-tree retrieval across fact↔chunk↔section↔doc-summary.
2. **Self-describing & introspectable** — per-subtree **capabilities manifest** + **answerable- questions index** (materialized from `hypothetical_q` reps).
3. **Verifiable by construction** — provenance (page+bbox) + `verification_status` + confidence; "verified-only" queries.
4. **Cross-lingual** — EN↔ES `translation` / `alt_phrasing` reps; query in one language hits the other.
5. **Access-aware projections** — per-node `sensitivity` → full vs masked view. **Toggleable** (`mask=false` ⇒ full); when on, only sensitive spans/values are masked, structure + non-PII content + provenance + traversal stay fully intact (D13).
6. **Time-travel & change-awareness** — version chain + validity → "as of date X" + "what changed" delta feed.
7. **Open representation system** — new `arep.rep_type`s are rows, never migrations.
8. **Hybrid scoped retrieval** — dense + lexical + structural(`ltree`) via RRF + `/api/rerank`, scoped to client/doc/section.

7. Cross-document merge (client-level view)

Intra-client only. `fact` knodes grouped by `attribute_key`; resolution = **confidence-weighted + flag** (highest-confidence source wins regardless of recency; disagreements set `conflict` + `needs_review`; all sources retained via `source_fact_ids`). No cross-client merge. The merged view is rebuildable from `knode` facts.

8. Serving API (downstream contract)

All endpoints `client_id`-scoped + RLS; `X-API-KEY`.

- `POST /api/v1/ingest` (SSE) — the pipeline.
- `GET /api/v1/clients/{id}/tree?doc_type=&version=&path=&depth=&mask=` — (sub)tree as nested JSON.
- `GET /api/v1/clients/{id}/facts?attribute_key=&verified_only=&mask=` — merged + per-doc facts.
- `GET /api/v1/clients/{id}/documents` — doc inventory + versions + deltas.
- `POST /api/v1/clients/{id}/search` — hybrid (dense+lexical+structural) scoped to client/doc/section; returns nodes + grounding (doc/page/bbox). The “find relevant documents / ask a question about a part” call.
- `GET /api/v1/nodes/{id}/provenance` — source doc/page/bbox + extractor + confidence.
- `GET /api/v1/clients/{id}/docs/{doc}/manifest` — capabilities manifest.
- `GET /api/v1/clients/{id}/docs/{doc}/answerable` — answerable-questions index.
- `GET /api/v1/clients/{id}/changes?since=` — version delta feed.
- `?mask=true|false` honored everywhere (D13).

9. Security & compliance

RLS-`FORCE` by `client_id` + per-connection GUC; `X-API-KEY` on all endpoints; PII gate keeps sensitive docs local; **masking projections** for least-privilege serving (toggleable); `di_decision_trace` audit; soft-delete + provenance retained for auditability; secrets via env / secret manager (no creds in code; no COIN/VDI creds here at all).

10. Repo layout (`~/document_intelligence`)

```
di/
  ocr_vision.py          # Azure Vision Read + fallback
  gate/                  # lang detect, stage0 anchors, classifier, presidio, routing
  extract/               # llm_extract.py (via retrieval), deterministic/ (per-
jurisdiction)
  subtree/               # build, chunk, context, arep gen, merge, versioning
  retrieval_client.py    # thin client for retrieval
/api/embed,/llm/complete,/rerank,/models
  db.py                  # asyncpg pool, ltree/pgvector bootstrap, RLS GUC, migrations
runner
  routers/               # ingest, clients, search, nodes
  ontology.py            # taxonomy (US/CA/MX), attribute_key catalog, per-type field
schemas
  migrations/            # 001_extensions(ltree), 002_documents, 003_knode_arep, 004_merge
...
reports/                 # requirements log + retrieval-api-requirements
docs/specs/              # this file
```

No vendoring of `get_llm.py` / `stellar_client.py` (model access is via the retrieval service). `db.py` patterns are adapted from retrieval (not a live dependency).

11. Build milestones

1. **M0 skeleton** — repo, config, `db.py`, migrations (ltree/pgvector bootstrap, RLS), retrieval client + `/api/models` handshake.
2. **M1 ingest core** — OCR → lang → gate(0–3) rules-only → deterministic extraction (passport + US/CA/MX checksummed IDs) → `di_documents` / `doc_version`.
3. **M2 knowledge subtree** — `knode` build + chunking + `/api/embed` + `context_prefix`; tree + hybrid scoped retrieval; provenance.
4. **M3 LLM path + arep** — `/api/llm/complete` classify/attribute-extract; full `arep` aids (incl. cross-lingual); answerable-questions + manifest.
5. **M4 merge + versioning** — `client_merged_fact` (confidence-weighted), version diff/reuse, delta feed.
6. **M5 capabilities + hardening** — masking projections (toggleable), classifier weak-supervision training from samples, eval harness, SSE polish.

12. Open / non-blocking

Q-M sample corpus (accelerates classifier); the retrieval-API additions must be deployed before M3 (LLM) and M2 (embed) — M0/M1 can proceed against rules + deterministic paths meanwhile.